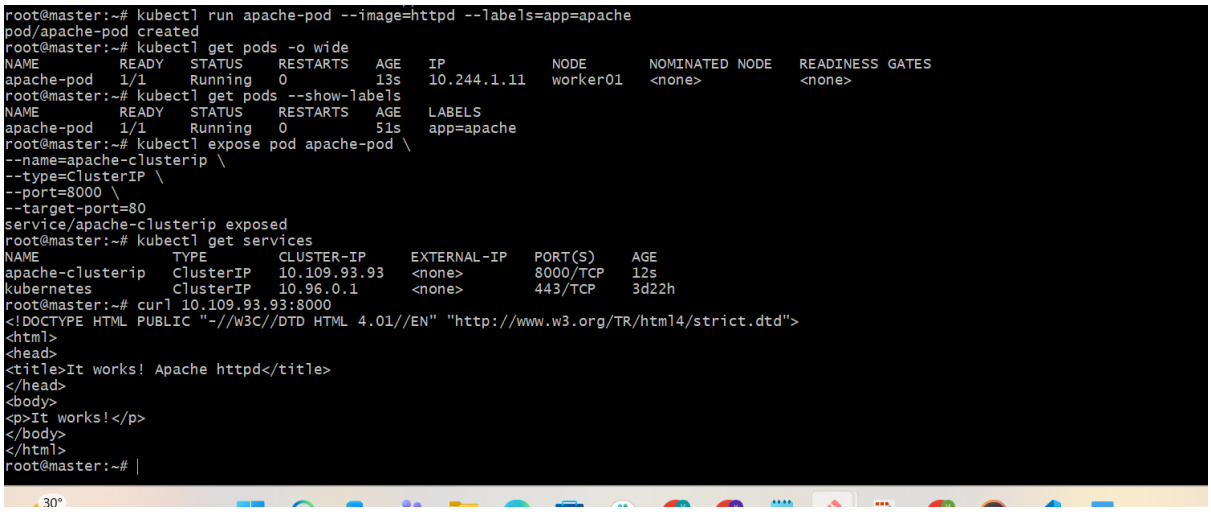
Tasks on K8S-03

1. **Create a ClusterIP service for an Apache web server pod.**

   Implementations steps:

   - Create an Apache pod using the following command:
     kubectl run apache-pod --image=httpd --labels=app=apache
   - Verify that the Apache pod is created:
     kubectl get pods
   - Expose the Apache pod using a ClusterIP service:
     kubectl expose pod apache-pod --name=apache-clusterip --type=ClusterIP --port=8000 --target-port=80
   - Verify the service:
     kubectl get services
   - Check the service details:
     kubectl describe service apache-clusterip
   - Verify that the service is connected to the Apache pod:
     kubectl get endpoints apache-clusterip
   - Test the service from inside the cluster:
     curl http://apache-clusterip:8000
   - Confirm that the Apache default web page is returned.



```
root@master:~# kubectl describe service apache-clusterip
Name:                     apache-clusterip
Namespace:                default
Labels:                   app=apache
Annotations:              <none>
Selector:                 app=apache
Type:                     ClusterIP
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.109.93.93
IPs:                      10.109.93.93
Port:                     <unset>  8000/TCP
TargetPort:               80/TCP
Endpoints:                10.244.1.11:80
Session Affinity:         None
Internal Traffic Policy:  Cluster
Events:                   <none>
```

```
root@worker01:~# curl 10.109.93.93:8000
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
root@worker01:~#
```

```
root@worker02:~# curl 10.109.93.93:8000
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
root@worker02:~#
```

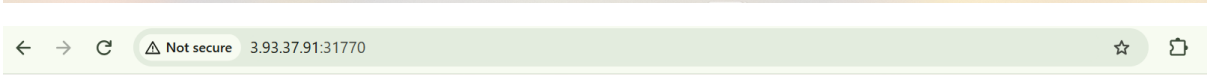
2. Expose an Nginx pod externally using a NodePort service.

Implementation steps:

- Create an Nginx pod:
  kubectl run nginx-pod --image=nginx --labels=app=nginx
- Verify that the pod is running:
  kubectl get pods
- Expose the Nginx pod using a NodePort service:
  kubectl expose pod nginx-pod --name=nginx-nodeport --type=NodePort --port=8000 --target-port=80
- Verify the NodePort service:
  kubectl get services
- Check the service details:
  kubectl describe service nginx-nodeport
- Note the NodePort number assigned by Kubernetes.
- Get the worker node IP address:
  kubectl get nodes -o wide

- Open the following URL in a browser: http://<Worker-Node-Public-IP>:<NodePort>
- If accessing through AWS EC2, allow the assigned NodePort in the worker node's security group.
- Verify that the Nginx welcome page is displayed.

```
root@master:~# kubectl run nginx-pod --image=nginx --labels=app=nginx
pod/nginx-pod created
root@master:~# kubectl get pods -o wide
NAME        READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
nginx-pod   1/1     Running   0          12s   10.244.1.12   worker01   <none>           <none>
root@master:~# kubectl expose pod nginx-pod \
--name=nginx-nodeport \
--type=NodePort \
--port=8000 \
--target-port=80
service/nginx-nodeport exposed
root@master:~# kubectl get services
NAME             TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
kubernetes       ClusterIP   10.96.0.1        <none>        443/TCP          3d23h
nginx-nodeport   NodePort    10.105.182.206   <none>        8000:31770/TCP   18s
root@master:~# kubectl describe service nginx-nodeport
Name:                     nginx-nodeport
Namespace:                default
Labels:                   app=nginx
Annotations:              <none>
Selector:                 app=nginx
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.105.182.206
IPs:                      10.105.182.206
Port:                     <unset>  8000/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  31770/TCP
Endpoints:                10.244.1.12:80
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
root@master:~# kubectl get nodes -o wide
NAME       STATUS   ROLES           AGE     VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION           CONTAINER-RUNTIME
master     Ready    control-plane   3d23h   v1.37.2   172.31.2.32    <none>        Ubuntu 26.04.1 LTS   7.0.0-1016-aws (amd64)   containerd://2.4.2
worker01   Ready    <none>          3d21h   v1.37.2   172.31.14.82   <none>        Ubuntu 26.04.1 LTS   7.0.0-1016-aws (amd64)   containerd://2.4.2
worker02   Ready    <none>          3d21h   v1.37.2   172.31.11.52   <none>        Ubuntu 26.04.1 LTS   7.0.0-1016-aws (amd64)   containerd://2.4.2
root@master:~#
```



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

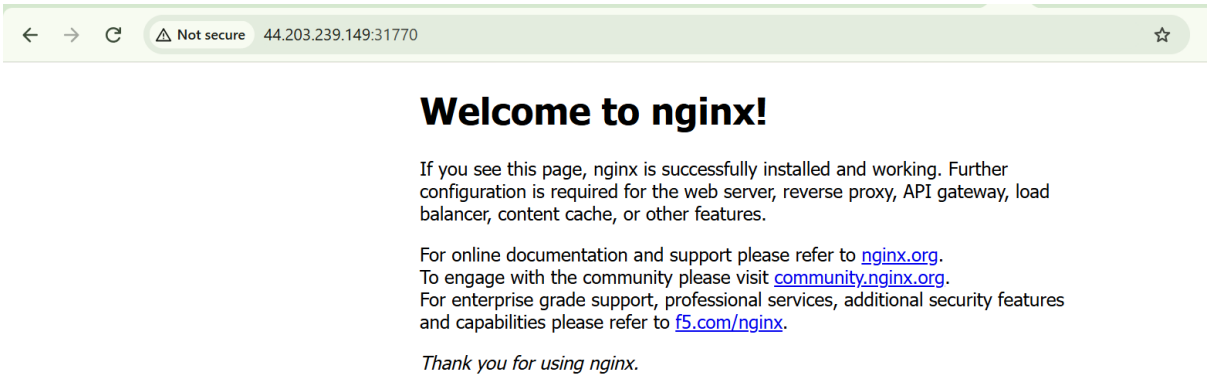


Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.



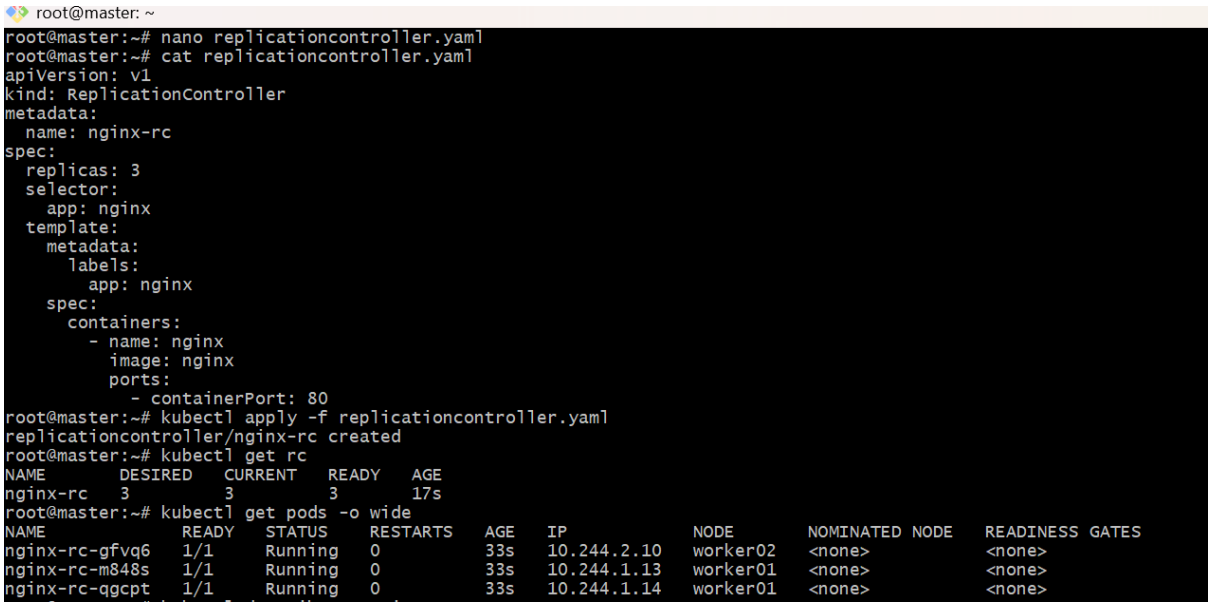
3. Deploy a ReplicationController to maintain 3 replicas of an Nginx pod.

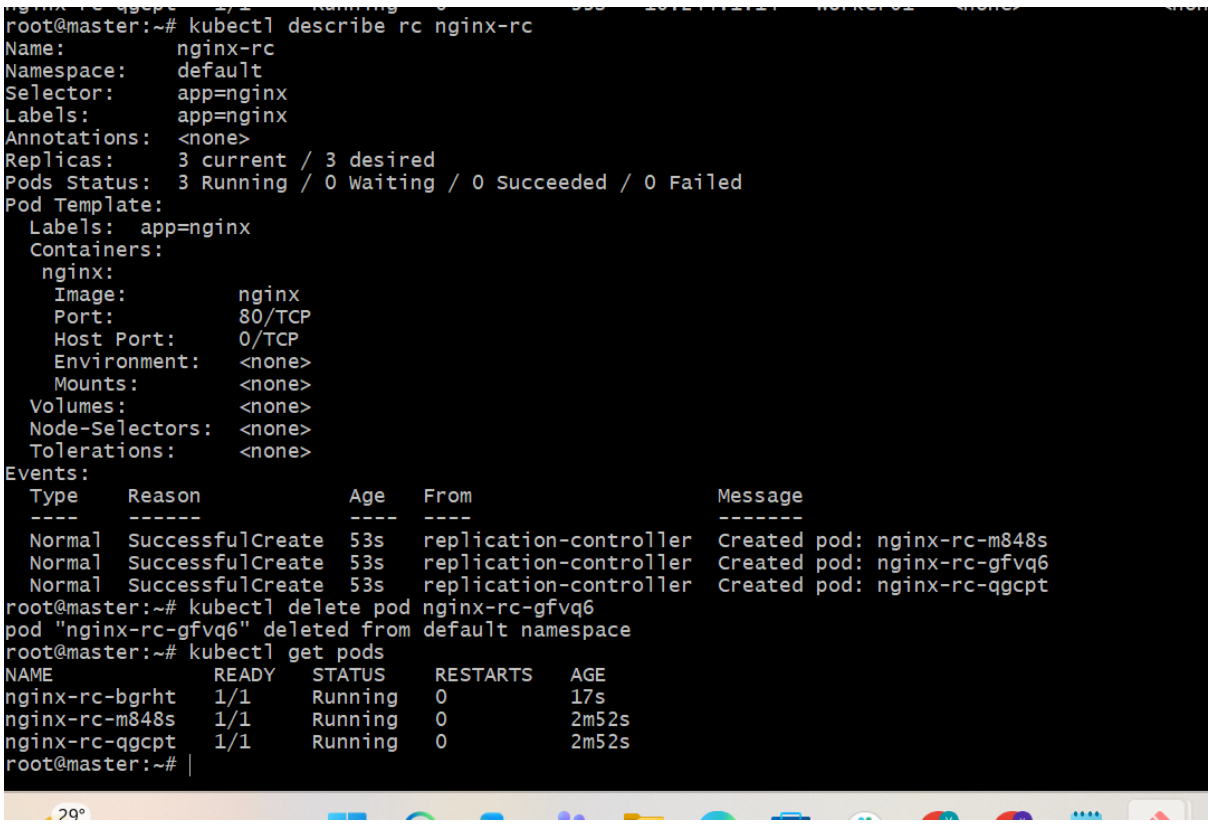
Implementation steps:

- Create a YAML file:
  vi replicationcontroller.yaml
- Add the following configuration:
  apiVersion: v1
  kind: ReplicationController
  metadata:
  name: nginx-rc
  spec:
  replicas: 3
  selector:
  app: nginx
  template:
  metadata:
  labels:
  app: nginx
  spec:
  containers:
  - name: nginx
  image: nginx
  ports:
  - containerPort: 80
- Save the file and create the ReplicationController:
  kubectl apply -f replicationcontroller.yaml
- Verify the ReplicationController:
  kubectl get rc
- Verify that three Nginx pods are created:
  kubectl get pods
- Check the details of the ReplicationController:
  kubectl describe rc nginx-rc
- Delete one of the Nginx pods:

kubectl delete pod <pod-name>

- Verify that the ReplicationController automatically creates a replacement pod:
  kubectl get pods

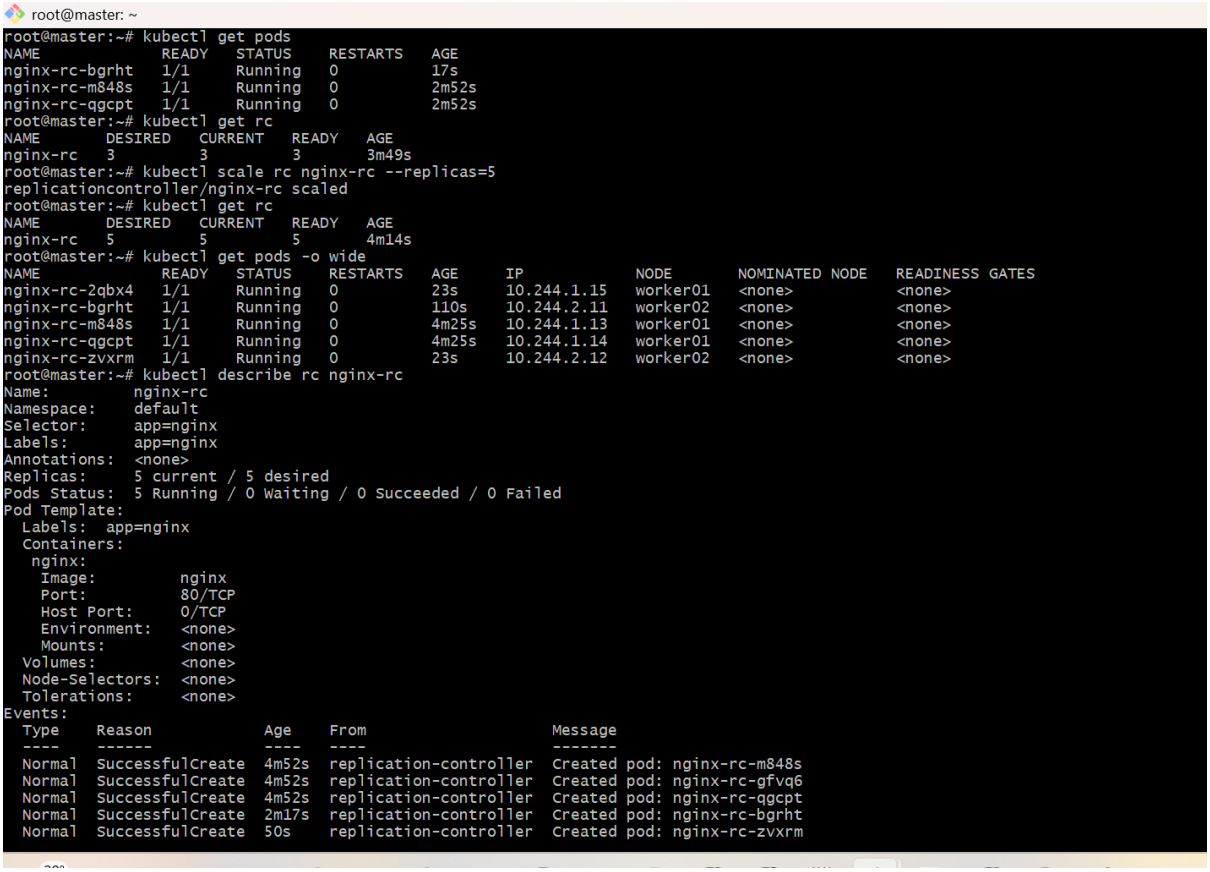




4. **Scale the ReplicationController from 3 replicas to 5 replicas.**

   Implementations steps:

   - Verify the existing ReplicationController:
     kubectl get rc
   - Check the currently running pods:
     kubectl get pods
   - Scale the ReplicationController from three to five replicas:
     kubectl scale rc nginx-rc --replicas=5
   - Verify the updated replica count:
     kubectl get rc
   - Verify that five Nginx pods are running:
     kubectl get pods
   - Check the ReplicationController details:
     kubectl describe rc nginx-rc
   - Update the replicas value to 5 in replicationcontroller.yaml so that the YAML file matches the current configuration.



5. **Create a ReplicaSet to manage pods based on multiple labels (prod and test).**

   Implementation steps:

- Create a YAML file for a pod with the prod label:
  vi prod-pod.yaml
- Add the following configuration:

  apiVersion: v1

  kind: Pod

  metadata:

  name: prod-pod

  labels:

  environment: prod

  spec:

  containers:

  name: nginx

  image: nginx

- Create another YAML file for a pod with the test label:
- vi test-pod.yaml
- Add the following configuration:
  apiVersion: v1
  kind: Pod
  metadata:
  name: test-pod
  labels:
  environment: test
  spec:
  containers:
  - name: nginx
  image: nginx
- Create both pods:
  kubectl apply -f prod-pod.yaml
  kubectl apply -f test-pod.yaml
- Verify the pods and their labels:
  kubectl get pods --show-labels
- Create a ReplicaSet YAML file:
  vi replicaset.yaml
- Add the following configuration:
  apiVersion: apps/v1
  kind: ReplicaSet
  metadata:
  name: nginx-rs

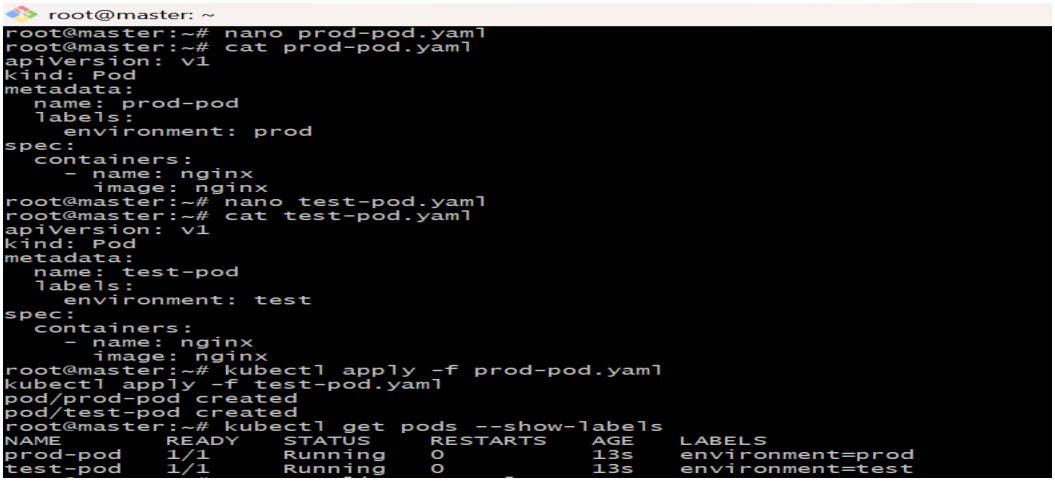
spec:
replicas: 2
selector:
matchExpressions:
- key: environment

operator: In
values:
- prod
- test

template:
metadata:
labels:
environment: prod
spec:
containers:
- name: nginx

image: nginx
ports:
- containerPort: 80

- Create the ReplicaSet:
  kubectl apply -f replicaset.yaml
- Verify the ReplicaSet:
  kubectl get rs
- Verify the pods managed by the ReplicaSet:
  kubectl get pods --show-labels
- Check the ReplicaSet details:
  kubectl describe rs nginx-rs
- Confirm that the ReplicaSet selector matches pods with either environment=prod or environment=test. It can adopt matching pods that are not already controlled by another workload.



```
root@master: ~
root@master:~# nano replicaset.yaml
root@master:~# cat replicaset.yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
  labels:
    app: nginx-rs
spec:
  replicas: 2
  selector:
    matchExpressions:
      - key: environment
        operator: In
        values:
          - prod
          - test
  template:
    metadata:
      labels:
        environment: prod
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
root@master:~# kubectl apply -f replicaset.yaml
replicaset.apps/nginx-rs created
```

```
root@master:~# kubectl get rs
NAME       DESIRED   CURRENT   READY   AGE
nginx-rs   2         2         2       12s
root@master:~# kubectl get pods --show-labels
NAME       READY   STATUS    RESTARTS   AGE     LABELS
prod-pod   1/1     Running   0          3m40s   environment=prod
test-pod   1/1     Running   0          3m40s   environment=test
root@master:~# kubectl describe rs nginx-rs
Name:         nginx-rs
Namespace:    default
Selector:     environment in (prod,test)
Labels:       app=nginx-rs
Annotations:  <none>
Replicas:     2 current / 2 desired
Pods Status:  2 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  environment=prod
  Containers:
   nginx:
    Image:         nginx
    Port:          80/TCP
    Host Port:     0/TCP
    Environment:   <none>
    Mounts:        <none>
  Volumes:         <none>
  Node-Selectors:  <none>
  Tolerations:     <none>
Events:            <none>
root@master:~#
```

6. Deploy a ReplicaSet that excludes pods with the label backend.

Implementation steps:

- Create a pod with the backend label:

kubectl run backend-pod --image=nginx --labels=app=prod,type=backend

- Verify the pod and its labels:
  kubectl get pods --show-labels
- Create a ReplicaSet YAML file:
  vi rs.yaml
- Add the following configuration:

```
apiVersion: apps/v1

kind: ReplicaSet

metadata:

 name: firstrc

 labels:

  appname: testapp


spec:

 replicas: 3

 selector:

  matchExpressions:

   - key: env

     operator: In

     values:

      - prod

      - test

   - key: type

     operator: NotIn   #ignore the pod with label as backend

     values:

      - backend

 template:

  metadata:

   name: firstpod

   labels:

    env: prod
```

```
spec:
  containers:
  - name: firstcontainer
    image: nginx
    env:
      - name: myname
```

- Create the ReplicaSet:
  kubectl apply -f rs.yaml
- Verify the ReplicaSet:
  kubectl get rs
- Verify the pods and their labels:
  kubectl get pods --show-labels
- Check the ReplicaSet details:
  kubectl describe rs firstrs
- Confirm that the ReplicaSet manages pods matching env=prod whose type label is not backend.
- Verify that backend-pod is not selected by the ReplicaSet.

```
root@master:~# nano firstpod.yaml
root@master:~# cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
  labels:
    env: prod
spec:
  containers:
  - name: nginx
    image: nginx
root@master:~# kubectl apply -f firstpod.yaml
pod/firstpod created
root@master:~# kubectl get pods
NAME       READY   STATUS    RESTARTS   AGE
firstpod   1/1     Running   0          8s
root@master:~# vi firstpod.yaml
root@master:~# cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secondpod
  labels:
    env: test
spec:
  containers:
  - name: nginx-test
    image: nginx
root@master:~# kubectl apply -f firstpod.yaml
pod/secondpod created
root@master:~# kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
firstpod    1/1     Running   0          81s
secondpod   1/1     Running   0          7s
root@master:~#
```

```
root@master:~# vi firstpod.yaml
root@master:~# cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: backendpod
  labels:
    env: prod
    type: backend
spec:
  containers:
  - name: nginx-prod
    image: nginx
root@master:~# kubectl apply -f firstpod.yaml
pod/backendpod created
root@master:~# kubectl get pods
NAME         READY   STATUS    RESTARTS   AGE
backendpod   1/1     Running   0          6s
firstpod     1/1     Running   0          3m51s
secondpod    1/1     Running   0          2m37s
root@master:~#
```

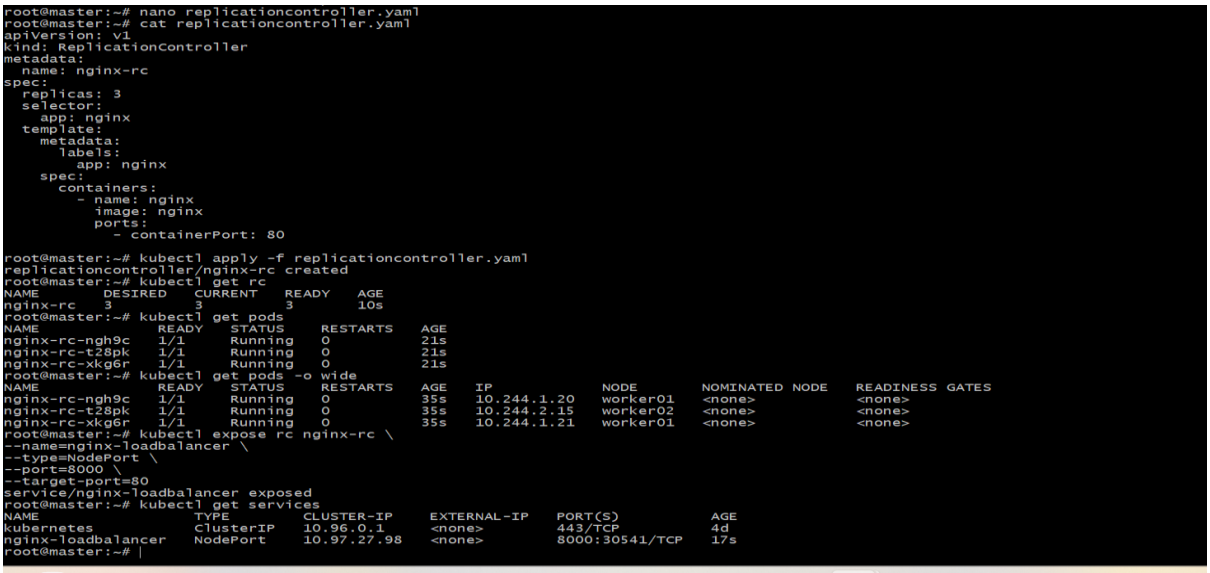
```
root@master:~# cat rs.yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: firstrc
  labels:
    appname: testapp

spec:
  replicas: 3
  selector:
    matchExpressions:
      - key: env
        operator: In
        values:
          - prod
          - test
      - key: type
        operator: NotIn   #ignore the pod with label as backend
        values:
          - backend
  template:
    metadata:
      name: firstpod
      labels:
        env: prod
    spec:
      containers:
      - name: firstcontainer
        image: nginx
        env:
          - name: myname
root@master:~# kubectl apply -f rs.yaml
replicaset.apps/firstrc created
root@master:~# kubectl get rs
NAME      DESIRED   CURRENT   READY   AGE
firstrc   3         3         3       12s
root@master:~# kubectl get pods --show-labels
NAME            READY   STATUS    RESTARTS   AGE     LABELS
backendpod      1/1     Running   0          87s     env=prod,type=backend
firstpod        1/1     Running   0          5m12s   env=prod
firstrc-wkhzn   1/1     Running   0          37s     env=prod
secondpod       1/1     Running   0          3m58s   env=test
root@master:~#
```

7. Test load balancing across multiple pods using a NodePort service.

Implementation steps:

- Before creating the service, make sure the ReplicationController has five Nginx pods running:
  kubectl get pods -l app=nginx
- If the standalone nginx-pod created in Question 2 is still running with the same app=nginx label, delete it after completing Question 2's test:
  kubectl delete pod nginx-pod
- This ensures that the service selects only the five ReplicationController pods.
- Expose the ReplicationController using a NodePort service:
  kubectl expose rc nginx-rc --name=nginx-loadbalancer --type=NodePort --port=8000 --target-port=80
- Verify the service:
  kubectl get services
- Check the service details:
  kubectl describe service nginx-loadbalancer
- Verify the service endpoints:
  kubectl get endpoints nginx-loadbalancer
- Confirm that the endpoints contain the IP addresses of the five Nginx pods.
- Get the worker node IP address:
  kubectl get nodes -o wide
- Note the NodePort number assigned to the service.
- Allow the NodePort in the AWS EC2 security group if the cluster is hosted on AWS.
- Access the application using the browser:
  http://<Worker-Node-Public-IP>:<NodePort>



```
root@master:~# kubectl describe service nginx-loadbalancer
Name:                     nginx-loadbalancer
Namespace:                default
Labels:                   app=nginx
Annotations:              <none>
Selector:                 app=nginx
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.97.27.98
IPs:                      10.97.27.98
Port:                     <unset>  8000/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  30541/TCP
Endpoints:                10.244.1.20:80,10.244.2.15:80,10.244.1.21:80
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
```

```
root@master:~# kubectl get nodes -o wide
NAME       STATUS   ROLES           AGE     VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION           CONTAINER-RUNTIME
master     Ready    control-plane   4d      v1.37.2   172.31.2.32    <none>        Ubuntu 26.04.1 LTS   7.0.0-1016-aws (amd64)   containerd://2.4.2
worker01   Ready    <none>          3d23h   v1.37.2   172.31.14.82   <none>        Ubuntu 26.04.1 LTS   7.0.0-1016-aws (amd64)   containerd://2.4.2
worker02   Ready    <none>          3d23h   v1.37.2   172.31.11.52   <none>        Ubuntu 26.04.1 LTS   7.0.0-1016-aws (amd64)   containerd://2.4.2
root@master:~# curl 44.211.183.145:30541
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@master:~#
```



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

8. Delete a ReplicationController without affecting the running pods.

Implementation steps:

- Verify the ReplicationController:
  kubectl get rc
- Verify the pods managed by the ReplicationController:
  kubectl get pods -o wide
- Delete the ReplicationController while keeping its pods running:
  kubectl delete rc nginx-rc --cascade=false

- Verify that the ReplicationController is deleted:
  kubectl get rc
- Verify that the previously created Nginx pods are still running:
  kubectl get pods
- Check the labels of the remaining pods:
  kubectl get pods --show-labels
- Delete one of the remaining Nginx pods:
  kubectl delete pod <pod-name>
- Verify the pods again:
  kubectl get pods
- Confirm that the deleted pod is not recreated because the ReplicationController no longer exists.

```
root@master:~# kubectl get rc
NAME       DESIRED   CURRENT   READY   AGE
nginx-rc   3         3         3       7m5s
root@master:~# kubectl get pods -o wide
NAME             READY   STATUS    RESTARTS   AGE     IP            NODE       NOMINATED NODE   READINESS GATES
nginx-rc-ngh9c   1/1     Running   0          7m20s   10.244.1.20   worker01   <none>           <none>
nginx-rc-t28pk   1/1     Running   0          7m20s   10.244.2.15   worker02   <none>           <none>
nginx-rc-xkg6r   1/1     Running   0          7m20s   10.244.1.21   worker01   <none>           <none>
root@master:~# kubectl delete rc nginx-rc --cascade=false
warning: --cascade=false is deprecated (boolean value) and can be replaced with --cascade=orphan.
replicationcontroller "nginx-rc" deleted from default namespace
root@master:~# kubectl get rc
No resources found in default namespace.
root@master:~# kubectl get pods
NAME             READY   STATUS    RESTARTS   AGE
nginx-rc-ngh9c   1/1     Running   0          10m
nginx-rc-t28pk   1/1     Running   0          10m
nginx-rc-xkg6r   1/1     Running   0          10m
root@master:~#
```

```
root@master:~# kubectl get pods
NAME             READY   STATUS    RESTARTS   AGE
nginx-rc-ngh9c   1/1     Running   0          10m
nginx-rc-t28pk   1/1     Running   0          10m
nginx-rc-xkg6r   1/1     Running   0          10m
root@master:~# kubectl describe pod nginx-rc-t28pk
Name:             nginx-rc-t28pk
Namespace:        default
Priority:         0
Service Account:  default
Node:             worker02/172.31.11.52
Start Time:       Mon, 09 Nov 2026 12:34:26 +0000
Labels:           app=nginx
Annotations:      <none>
Status:           Running
IP:               10.244.2.15
IPs:
  IP:  10.244.2.15
Containers:
  nginx:
    Container ID:   containerd://521f31bd4636fd6e2564dd916991ffd04f027ca676fd58c0a437af0d0e76e6c0
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Mon, 09 Nov 2026 12:34:27 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-kt97r (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
```